CE 502 · ADVANCED TOPICS IN DATABASE SYSTEMS

Week 02

Backup, Recovery & Replication

How a database administrator makes sure data is never lost, and how a database survives the loss of a whole server.

Dr. Afan HASAN · [Semester]

THIS WEEK

- 2-1 **Database Backups** — 45 minutes
- 2-2 **Replication and High Availability** — 45 minutes
- 2-3 **Database Recovery** — 45 minutes
- P2 **Practice: backup and restore** — 60 minutes · PostgreSQL lab

Recap: The Transaction Log

- Every change is first written to a sequential **log**: the WAL in PostgreSQL, the redo log in Oracle, the transaction log in SQL Server.
- **Write-ahead rule:** a change reaches the log on disk before the changed data page does.
- At **COMMIT** the log is forced to disk; with the log the DBMS can undo a failed transaction and redo committed work after a crash.

THIS WEEK THE LOG GETS TWO NEW JOBS

1. **Recovery from backup:** restore an old copy, then replay the logs.
2. **Replication:** ship the log to a second server and replay it there.

Key point: crash recovery needs the data files and the log to survive. This week deals with failures that destroy them.

CE 502 · WEEK 2 · LESSON

2-1

Database Backups

Lesson 1 of 3

In this lesson

01 Why databases need backups

02 How often to back up

03 Full, differential and incremental backups

04 Cold and warm, logical and physical backups

45 minutes

Builds on Week 1: the transaction log

Objectives

By the end of this lesson, you will be able to:

01 **Explain** why databases need backups and what causes data loss

Checked in: Quiz 2 · questions 1–2

02 **Decide** how often a database should be backed up

Checked in: Quiz 2 · question 3 and the scenario task

03 **Distinguish** full, differential and incremental backups and their restore chains

Checked in: Quiz 2 · questions 4–6 and Practice 2

04 **Compare** cold and warm backups, and logical and physical backups

Checked in: Quiz 2 · questions 7–8

Look for the objective chip in the top-right corner of each slide: it shows which goal that slide serves.

Objective 01

Why Back Up a Database?

- A **backup** is a copy of the database taken at a point in time and stored **separately** from the original, so it survives when the original is lost.

- The first rule of database administration is simple: never lose data. Backups are the last line of defence.

- A copy on the same disk as the database is **not** a backup: one failure destroys both.

Five causes of data loss

- Hardware failure
- Software bug or corruption
- Human error
- Ransomware and attacks
- Site disaster: fire, flood

Key point: crash recovery from Week 1 only works if the disk and the log survive. Backups protect against losing them.

Common Causes of Data Loss

A backup plan must protect against every cause, not only against hardware failure.

Cause	Example	Does a standby server help?
Hardware failure	A disk or storage array fails	Yes, it takes over
Corruption	A bug writes bad data into a file	Often no: bad data is copied
Human error	DELETE without a WHERE clause	No: the delete is replicated
Ransomware	Malware encrypts the database files	No: needs an isolated copy
Site disaster	Fire or flood destroys the data centre	Only if it is in another place

Read the last column. A second server helps with only two of the five causes. The rest need a backup.

Key point: crash recovery handles a crash; only a separate copy handles the loss of the disk, the data or the site.

Objective 02

How Often Should a Database Be Backed Up?

There is no single correct frequency. It depends on four factors.

Size

A larger database takes longer to copy and to restore.

Users

More users means more work lost per hour of data loss.

Course catalogue

50 MB, changed a few times a term.

Plan: weekly full backup.

Rate of change

Data changed every second needs more frequent protection.

Criticality

How badly the organisation suffers if the data is lost.

Online shop orders

2 TB, thousands of orders an hour.

Plan: nightly backups plus continuous log archiving.

← **Same DBMS, very different plans:** backup frequency is a business decision as much as a technical one.

Objective 01

A backup you have never restored is not a backup.

Backups fail silently: a job that stopped months ago, a missing log file, an encryption key nobody kept. The only proof that a backup works is a successful test restore.

- Rule of thumb: test-restore every critical database on a fixed schedule, and time it.

Objective 03

Full, Differential and Incremental Backups

Full

Copies **the entire database**.

Backup: slowest and largest

Restore needs: this file only

Best for: the weekly baseline of every plan

Differential

Copies what changed since the **last full** backup.

Backup: grows larger each day

Restore needs: full + latest differential

Best for: fast, simple restores

Incremental

Copies what changed since the **last backup of any kind**.

Backup: fastest and smallest

Restore needs: full + every incremental, in order

Best for: large databases, short backup windows

⚠ **Common mistake:** some tools and courses call any partial backup “incremental”. Always ask: changes since the last *full* backup, or since the last backup of *any* kind?

Example: Restoring Thursday Night

A shop takes a full backup every Sunday night. The database fails on Friday morning. Which files are needed under each plan?

Night	Plan A: differentials	Plan B: incrementals
Sunday	Full	Full
Monday	Diff (Mon)	Incr (Mon)
Tuesday	Diff (Mon–Tue)	Incr (Tue)
Wednesday	Diff (Mon–Wed)	Incr (Wed)
Thursday	Diff (Mon–Thu)	Incr (Thu)
Restore	Sun full + Thu diff: 2 files	Sun full + 4 incrementals: 5 files

In practice

PostgreSQL 17 added incremental physical backups: **pg_basebackup --incremental**, merged before restore with **pg_combinebackup**.

← **Read this row first.** Plan B needs five files, and one damaged file breaks the whole chain.

Cold and Warm (Online) Backups

Backups also differ in whether the database keeps running while they are taken.

Cold backup

The database is **shut down** and its files are copied.

+ Simple; the copy is consistent by itself

– Requires downtime for users

Best for: small systems with a maintenance window

Warm (online) backup

The backup runs **while users keep working**.

+ No downtime

– Longer; needs the logs written during the copy

Best for: production systems that run 24/7

⚠ **Common mistake:** names vary. Many products call a warm backup an “online” or “hot” backup, and some courses use “hot” only for a standby server.

Objective 04

Logical and Physical Backups

Backups also differ in the form of the copy: SQL that re-creates the data, or the data files themselves.

	Logical (pg_dump)	Physical (pg_basebackup)
What is copied	Definitions and rows, as SQL	The data files, block by block
Portable	Yes: across versions	No: same version and platform
Granularity	One table or schema possible	Whole database cluster
Speed on large DBs	Slow to back up and restore	Fast: copies files directly
Point-in-time recovery	No	Yes, with archived logs

In practice

Logical: **pg_dump**, **mysqldump**. Physical: **pg_basebackup**, **Oracle RMAN**, SQL Server **BACKUP DATABASE**.

Critical: only a physical backup plus archived logs can reach an exact minute.

Storing Backups Safely

- A backup is only useful if it survives the event that destroyed the database.
- Keep at least one copy **off-site**, in another building or cloud region.
- Keep one copy **immutable** or offline, so ransomware cannot delete or encrypt it.
- **Encrypt** backups: a stolen backup exposes the same personal data as the database.

THE 3-2-1 RULE

3 copies of the data

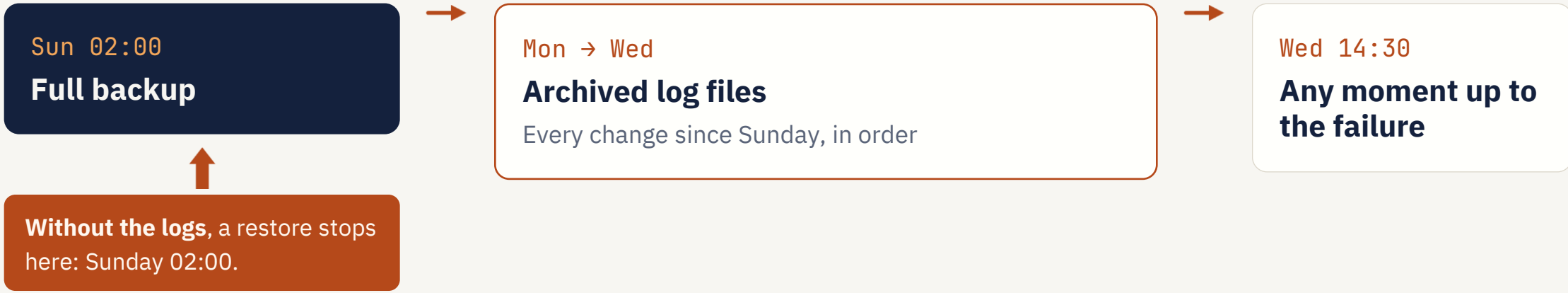
2 different types of storage

1 copy off-site

In practice

Many teams now use **3-2-1-1-0**: add one immutable copy and zero errors after a test restore.

Always Back Up the Logs as Well



• Saving the logs written since the backup is called **log archiving**. Replayed on top of the backup, they reach any later moment.

In practice

PostgreSQL: archive_mode = on plus an archive command copies each completed WAL file. Oracle calls this ARCHIVELOG mode; SQL Server uses transaction log backups.

Objective 02

Case Study: An Online Bookshop

THE SITUATION

A regional online bookshop runs an 80 GB PostgreSQL database. Orders arrive 24 hours a day, with a quiet period from 02:00 to 04:00.

The owner says:

> “We can lose at most **one hour** of orders. We cannot afford a second server.”

The budget includes one cloud storage account.

YOUR TASK · PAIRS · 8 MIN

1. Cold or warm backups? Why?
2. Which backup types and schedule?
3. How do you meet the one-hour limit?
4. Where and how do you store the copies?

Objective 03

Check Your Understanding

A company takes a full backup on Sunday and an **incremental** backup every other night. The database fails on Friday morning. Which files are needed to restore Thursday night's state?

A Sunday's full backup only

B Sunday's full + Thursday's incremental

C Sunday's full + Mon, Tue, Wed and Thu incrementals

D Thursday's incremental only

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Backup
A copy of the database at a point in time, stored separately from it.

Full backup
A copy of the entire database that can be restored on its own.

Differential backup
A copy of all changes since the last full backup.

Incremental backup
A copy of all changes since the last backup of any kind.

Restore chain
The ordered set of backup files needed to rebuild a database.

Cold / warm backup
Taken with the database offline / while it stays online.

Logical backup
SQL statements that re-create the data, e.g. pg_dump.

Physical backup
A copy of the data files themselves, e.g. pg_basebackup.

Summary

In this lesson, you learned how to:

- **01** Explain why databases need backups and what causes data loss
- **02** Decide how often a database should be backed up
- **03** Distinguish full, differential and incremental backups and their restore chains
- **04** Compare cold and warm backups, and logical and physical backups

Key takeaway: back up the database **and** the logs, store copies elsewhere, and prove it works with a timed test restore.

Next lesson

2-2 Replication and High Availability

CE 502 · WEEK 2 · LESSON

2-2

Replication and High Availability

Lesson 2 of 3

In this lesson

01 What replication is and how it uses the log

02 Synchronous and asynchronous replication

03 Failover, switchover and split brain

04 Why replication does not replace backups

45 minutes

Builds on Lesson 2-1 and Week 1

Objectives

By the end of this lesson, you will be able to:

01 **Explain** how replication uses the transaction log

Checked in: Quiz 2 · questions 9–10

02 **Compare** synchronous and asynchronous replication

Checked in: Quiz 2 · questions 11–12 and the scenario task

03 **Describe** failover, switchover and split brain

Checked in: Quiz 2 · question 13

04 **Explain** why replication does not replace backups

Checked in: Quiz 2 · question 14

Look for the objective chip in the top-right corner of each slide: it shows which goal that slide serves.

Objective 01

What Is Replication?

Replication keeps a second, almost identical copy of a database on another server, updated continuously.

- The main server is the **primary**; the copy is the **standby** (also called a replica or secondary).
- The primary sends its **transaction log records** to the standby as it writes them.
- The standby **replays** the records, just as crash recovery would, and stays a few moments behind.

An analogy

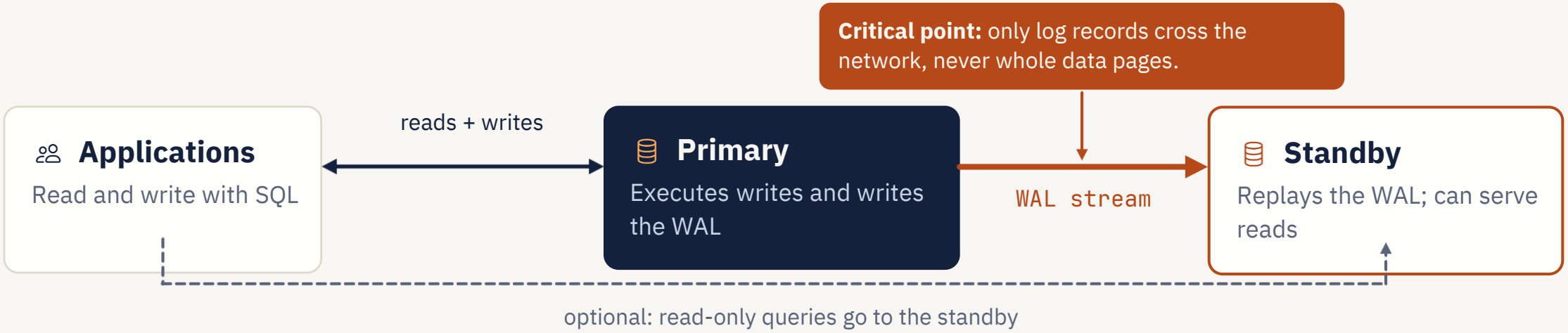
A teacher writes notes on the board; a student copies each line into a notebook as soon as it appears.

If the board is wiped, the notebook has everything up to the last line copied.

Key point: Replication = shipping the log + replaying it. Nothing new is needed beyond the log you met in Week 1.

Objective 01

Primary–Standby Architecture



- The standby starts from a full copy of the primary, then keeps applying the log records it receives.
- The delay between a commit on the primary and its replay on the standby is the **replication lag**.

Objective 01

What Is a Standby Used For?

Taking over

If the primary fails, the standby is **promoted** and becomes the new primary.

Downtime falls from hours to seconds or minutes.

Read-only queries

Reports and searches can run on the standby.

This takes load off the primary; a standby used this way is a **hot standby**.

Backups

Backups can be taken from the standby instead of the primary.

The primary is not slowed down by the backup.

In practice

In PostgreSQL, a standby that accepts read-only queries is enabled with hot_standby = on (the default). Queries there may see data that is a fraction of a second old.

Objective 01

Two Modes of a Standby: Recovery and Read-Only

A standby never finishes recovery, but it can answer queries at the same time.

Recovery mode

The standby is permanently “recovering”: it receives log records and replays them.

No SQL writes are possible.

Read-only mode

While recovering, it also answers **SELECT** queries.

Any INSERT, UPDATE or DELETE is rejected.

```
standby=> SELECT bal FROM accounts
WHERE id = 1;
bal: 250.00
standby=> UPDATE accounts SET bal = 0
WHERE id = 1;
ERROR: cannot execute UPDATE
in a read-only transaction
```

← Only the primary may change data. Every change on the standby must come from the log.

Objective 02

Synchronous vs Asynchronous Replication

The difference is **when** the primary tells the application that a transaction has committed.

	Synchronous	Asynchronous
Commit waits for	Standby confirms it received the log	Nothing: the primary does not wait
Commit latency	Higher: one network round trip	Lowest
Data loss on failover	None for confirmed commits	Recent commits may be lost
If the standby is down	Commits wait or stop	Primary continues normally
Typical use	Banking, payments, orders	Read scaling, remote disaster sites

← Synchronous costs **one network round trip per commit**: about 1 ms in the same data centre, 50–100 ms between continents.

Objective 02

Scenario: A Bank and a Social Feed

Two companies, one question: which replication mode should each use?

Bank · payments team

A customer transfers €5,000. The app says “Transfer complete”. One second later, the primary server fails.

Social app · feed team

Users post 20,000 likes and comments per second worldwide. A slow app loses users.

YOUR TASK · PAIRS · 5 MIN

1. Which mode should each system use?
2. With async, what exactly could the bank lose?
3. What does sync cost the social app?

There is no “best” mode: decide from what **one lost commit** costs the business.

Objective 03

Failover: Promoting the Standby

1

Detect

Health checks see that the primary is not responding.

2

Fence

Make sure the old primary can no longer accept writes.

3

Promote

The standby leaves recovery and accepts writes.

4

Redirect

Applications connect to the new primary.

5

Rebuild

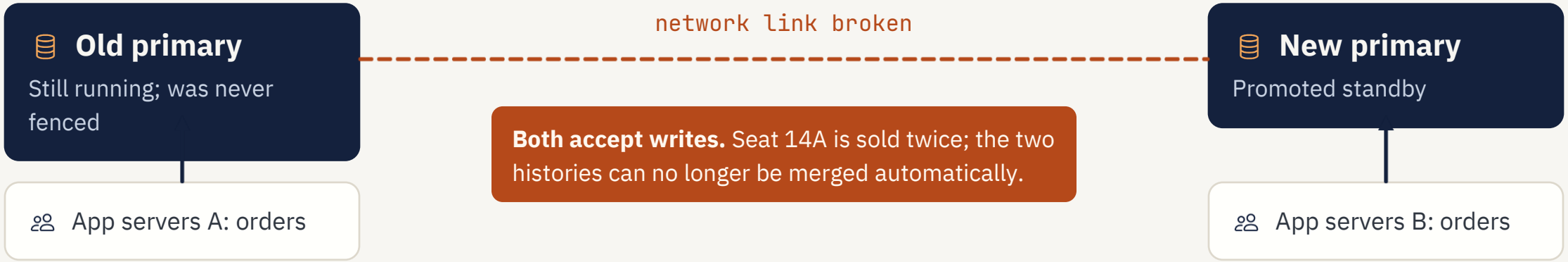
Create a new standby so protection is restored.

Skipping this step causes split brain.

- **Failover:** unplanned, after the primary fails.
- **Switchover:** planned role swap, e.g. for maintenance; no data is lost.

Objective 03

Split Brain: Two Primaries at Once



Fencing (STONITH)

Power off or isolate the old primary before promoting.

Quorum

Act only with a majority of nodes, e.g. 2 of 3. Raft works this way.

Witness

A small third node breaks ties between two data centres.

Objective 04

Replication is not a backup.

A standby copies every change within milliseconds, including the wrong ones.

```
14:31:07 primary DROP TABLE orders;
14:31:07 standby replays: DROP TABLE orders;
```

Same second. Only a backup plus archived logs can return to 14:31:06.

Objective 01

Replication in Real Products

Every major product replicates by shipping its transaction log; only the names differ.

Product	Replication feature	What is shipped
PostgreSQL	Streaming replication	WAL records
MySQL	Source–replica replication	Binary log events
SQL Server	Always On availability groups	Transaction log blocks
Oracle Database	Data Guard	Redo data
Cloud, e.g. Amazon RDS	Multi-AZ deployment	Managed; automatic failover

← Different names, one idea: **ship the log, replay it.**

- All support synchronous and asynchronous modes and read-only standbys; they differ in how failover is automated.

Objective 02

Check Your Understanding

A primary uses **asynchronous** replication. Its replication lag is about 0.5 seconds when the primary crashes, and the standby is promoted. What happens to the data?

A Nothing is lost: the standby has every committed transaction

B Commits from the last ≈0.5 s before the crash may be lost

C All transactions since the last backup are lost

D The standby cannot be promoted after an async failure

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Replication

Keeping a copy of a database up to date on another server by shipping and replaying its log.

Standby

A server that replays the primary's log and can take over; also replica or secondary.

Replication lag

The delay between a commit on the primary and its replay on the standby.

Failover / switchover

Promoting a standby after a failure / swapping roles in a planned way.

Primary

The server that accepts writes and sends its log to the standbys.

High availability

Keeping a service usable with very little downtime, even when a server fails.

Synchronous replication

A commit waits until a standby confirms it received the log records.

Split brain

Two servers act as primary at once and their data diverges.

Summary

In this lesson, you learned how to:

- **01** Explain how replication uses the transaction log
- **02** Compare synchronous and asynchronous replication
- **03** Describe failover, switchover and split brain
- **04** Explain why replication does not replace backups

Key takeaway: replication keeps a live copy for availability; backups and archived logs keep history. You need both.

Next lesson

2-3 Database Recovery

CE 502 · WEEK 2 · LESSON

2-3

Database Recovery

Lesson 3 of 3

In this lesson

01 What recovery must achieve

02 The six-step recovery procedure

03 Point-in-time recovery

04 RPO, RTO and designing a strategy

45 minutes

Builds on Lessons 2-1 and 2-2

Objectives

By the end of this lesson, you will be able to:

01 **Explain** what recovery must achieve, and distinguish crash recovery from media recovery

Checked in: Quiz 2 · question 15

02 **Apply** the six-step recovery procedure and point-in-time recovery

Checked in: Practice 2 · Part B

03 **Define** RPO and RTO, and relate them to backup and replication techniques

Checked in: Quiz 2 · question 16

04 **Design** and test a backup and recovery strategy for given requirements

Checked in: Quiz 2 · design question and the case study

Objective O2 is practised hands-on in Practice 2 at the end of this week.

Objective 01

What Recovery Must Achieve

- Recovery brings a damaged or lost database back to a **consistent** state that is as close as possible to the moment just before the failure.
- Consistent means every committed transaction is present and no half-finished transaction is.
- A backup alone only brings back the past; the **logs** written after the backup bring the database forward.

Two inputs, one result

Backup: the database at a past moment

Logs: every change since that moment

Result: consistent database just before the failure

Key point: recovery = restore the backup + roll forward with the logs.

Objective 01

Crash Recovery and Media Recovery

Crash recovery

Cause: power cut, server or DBMS crash

What survives: data files and the log on disk

Starts from: the files left on the server

Who starts it: the DBMS, automatically at restart

Typical time: seconds to minutes

Media recovery

Cause: disk loss, corruption, human error

What survives: only the backups and archived logs

Starts from: the most recent usable backup

Who starts it: the DBA, following a procedure

Typical time: minutes to hours

⚠ **Common mistake:** “The DBMS recovers from crashes, so we do not need backups.” Crash recovery cannot help when the disk that holds the log is gone.

Objective 02

Recovering a Database in Six Steps

After a disk failure or corruption, the DBA follows a fixed procedure. Each step depends on the one before it.

01

Find the most recent **complete backup**

02

Restore that backup to the database server

03

Establish the **point in time** of the failure

04

Find the last **log file** written before the failure

05

Find the last **transaction** before the failure in that log

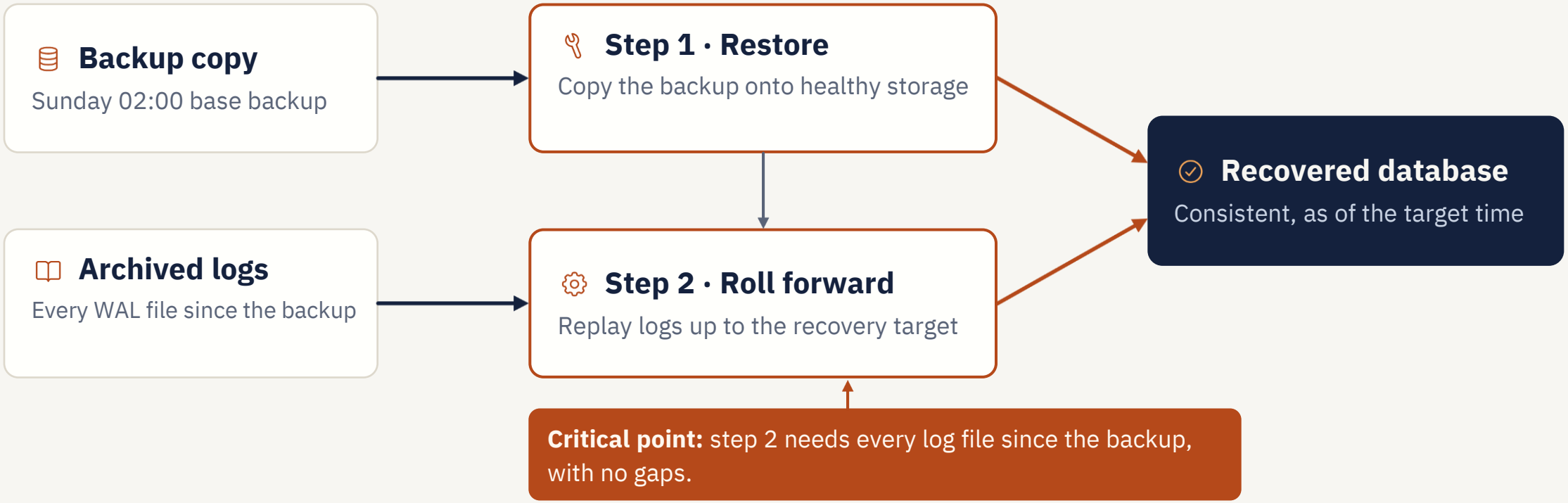
06

Apply the logs up to the last commit before the failure

Key point: steps 1–2 bring back the past; steps 3–6 replay the log to bring the database forward to the moment before the failure.

Objective 02

Standard Recovery: Restore, Then Roll Forward

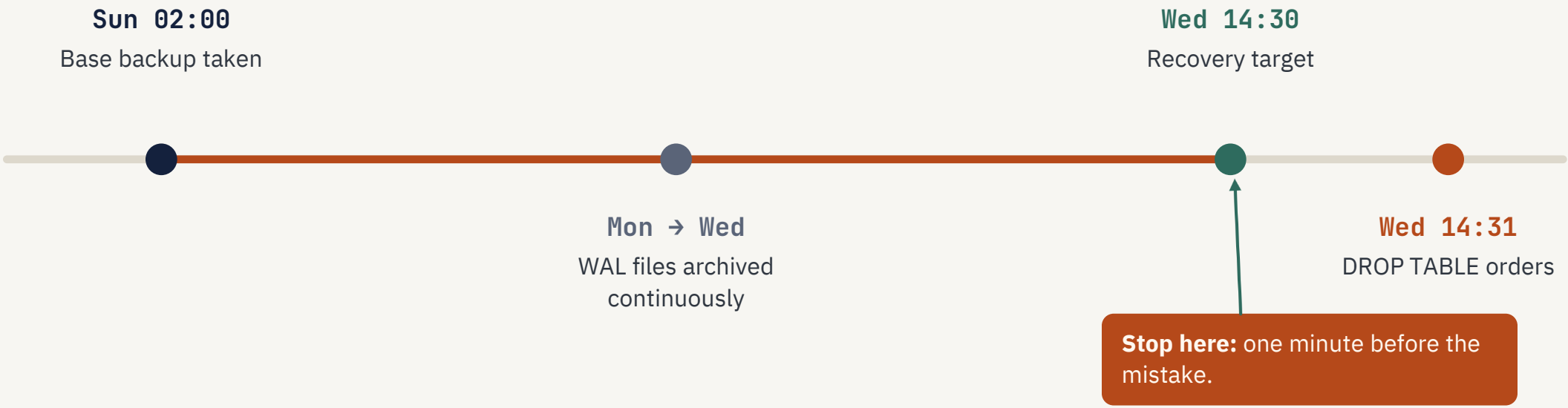


- The database stays offline during both steps; applications reconnect only when the recovered database has been checked.

Objective 02

Point-in-Time Recovery

Someone drops the orders table on Wednesday at 14:31. Point-in-time recovery brings it back without losing the morning’s work.



- Restore the base backup, replay archived logs, and **stop at the recovery target**: a time, a transaction or a log position.
- Everything committed after the target is lost, so choose the target as close to the mistake as possible.

Objective 03

RPO and RTO: The Two Numbers Behind Every Plan

RPO

Recovery Point Objective

How much data can we afford to lose?

The maximum acceptable data loss, measured as time before the failure.

Example: “at most 5 minutes of orders”.

RTO

Recovery Time Objective

How long can we afford to be down?

The maximum acceptable time from the failure until the service works again.

Example: “back online within 1 hour”.

Key point: design starts from RPO and RTO agreed with the business, not from the tools you happen to know.

Objective 03

How Each Technique Changes RPO and RTO

Each layer of protection reduces possible data loss or downtime. Figures are typical orders of magnitude, not guarantees.

Strategy	Typical RPO	Typical RTO
Nightly backup only	Up to 24 hours	Hours (full restore)
+ continuous log archiving	Minutes	Hours (restore + replay)
+ asynchronous standby	Seconds (the lag)	Minutes (failover)
+ synchronous standby	About zero	Seconds to minutes

← **Log archiving** is what moves RPO from a day to minutes.

← **Careful:** a standby helps only with server failure. A mistaken DELETE still needs PITR, with its longer RTO.

Key point: combine layers: replication for fast failover, backups with log archiving for everything replication cannot fix.

Objective 04

Case Study: The University Registry

THE SITUATION

The registry database holds 20 years of grades for 15,000 students. It is 400 GB and changes all day during exam weeks, but little in summer.

The rector’s office says:

> “We can accept losing at most **5 minutes** of grade entries, and the system must be back within **1 hour**.”

The budget allows one extra server and off-site cloud storage.

YOUR TASK · PAIRS · 8 MIN

1. What are the RPO and RTO in this scenario?
2. Which backup types and schedule would you choose?
3. Is nightly backup alone enough? What must you add?
4. How will you prove the plan works?

Objective 04

Designing a Backup and Recovery Strategy

A strategy is a chain of decisions. Each step answers a question raised by the one before it.

01

Agree **RPO and RTO** with the data owners

02

Choose **backup types** and a schedule

03

Add **log archiving** and **replication** to meet the targets

04

Store copies **safely**: 3-2-1, immutable, encrypted

05

Write a **runbook** and **test** it on a schedule

In practice

Tools such as pgBackRest and Barman (PostgreSQL), RMAN (Oracle) and the SQL Server Agent automate steps 2–4: scheduling, archiving, retention and verification.

Objective 04

Test Your Restores

- Restore critical databases on a **fixed schedule**, for example monthly, into a separate test server.
- Include a **point-in-time recovery** and a standby **failover** in the rehearsal, not only a plain restore.
- Compare the measured time with the **RTO**: if it is longer, the plan does not meet its target.

A restore test records

- Date, database and backup set used
- Time to restore and to replay the logs
- Row counts or checksums compared
- Problems found and how they were fixed
- Sign-off by the data owner

Key point: an untested recovery plan is a hypothesis, not a plan.

Objective 02

Check Your Understanding

A company takes a backup every night at 01:00, archives its WAL continuously and has an asynchronous standby. At 16:45 an employee deletes the **customers** table. What is the best possible outcome?

A Restore the 01:00 backup and accept losing almost 16 hours

B Restore the 01:00 backup and replay the WAL to just before 16:45

C Fail over to the standby, which still has the table

D Nothing can be done without a cold backup

Think for 30 seconds, then vote: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Crash recovery

Automatic replay of the on-server log when the DBMS restarts after a crash.

Roll forward

Replaying logged changes on top of a restored backup.

Recovery target

The time, transaction or log position at which replay stops.

RTO

Recovery time objective: the maximum acceptable downtime.

Media recovery

Restoring a backup and replaying archived logs after data files are lost or damaged.

Point-in-time recovery

Recovering a database to a chosen moment, such as just before an error.

RPO

Recovery point objective: the maximum acceptable data loss, measured as time.

Runbook

A written, step-by-step recovery procedure that others can follow.

Summary

In this lesson, you learned how to:

- **01** Explain what recovery must achieve, and distinguish crash recovery from media recovery
- **02** Apply the six-step recovery procedure and point-in-time recovery
- **03** Define RPO and RTO, and relate them to backup and replication techniques
- **04** Design and test a backup and recovery strategy for given requirements

Key takeaway: recovery = backup + unbroken log chain + a recovery target, designed from RPO and RTO and proven by timed tests.

Next

Practice 2: Backup and Restore

Overview

Practice 2: You Are the DBA During an Incident

You protect a small shop database, simulate a mistake, and bring the lost data back.

Setup	Create the database **shop** with an orders table of 50,000 rows.	–
Part A	Logical backup with pg_dump, an accidental DELETE, and a timed restore with pg_restore.	Lessons 2-1, 2-3
Part B · bonus	Base backup with WAL archiving, a DROP TABLE, and point-in-time recovery.	Lessons 2-1, 2-3
Report	Times, row counts, measured RTO and RPO, and a short recovery runbook.	Lesson 2-3

You need: Docker, the course PostgreSQL 16 container, and the **week02** folder of the course kit.

Taking and Restoring a Logical Backup

```
-- terminal
pg_dump -U postgres -Fc shop > shop.dump
psql shop -c "DELETE FROM orders WHERE id%2=0;"
createdb -U postgres shop_restored
time pg_restore -U postgres -d shop_restored shop.dump
```

1. Dump the whole database in custom format (-Fc).
2. Simulate a mistake: delete half the orders.
3. Create an empty database to restore into.

← **4 · Critical:** restore and **time** it. The measured time is your real RTO.

⚠ **Common mistake:** restoring over the damaged database. Always restore into a new database first, check it, then switch.

Part B · bonus

Point-in-Time Recovery in PostgreSQL

1

```
-- postgresql.conf
wal_level = replica
archive_mode = on
archive_command = 'cp %p /archive/%f'
```

2

```
-- terminal
pg_basebackup -U postgres -D /backup/base -Fp -P
psql shop -c "SELECT now();" -- note the time
psql shop -c "DROP TABLE orders;"
```

3

```
-- recovery: postgresql.conf + empty recovery.signal
restore_command = 'cp /archive/%f %p'
recovery_target_time = '<time noted above>'
touch /backup/pitr/recovery.signal
```

1 Check that WAL archiving is on (it is in the course container).

2 Take a base backup, add rows, note the time, drop the table.

3 Copy the base backup to a new directory, add these settings, start it as a second server.

← **Critical:** recovery_target_time must be **before** the mistake, or the DROP is replayed too.

What to Hand In

A report of one to two pages, submitted as a PDF before the Week 3 class.

Report item	Points
Part A: backup time, dump file size and restore time	2
Part A: row counts before and after, with one sentence of interpretation	2
Your measured RTO, and the RPO of a nightly pg_dump	2
A recovery runbook of 5–8 numbered steps	2
Bonus Part B: recovered table and the recovery lines from the server log	+2
Total	10

Key point: every number in the report needs one sentence that says what it proves.



Papers to Read

Three required papers and three optional ones on replication, cloud recovery and consensus.

Before the Week 3 class

Half-page note per required paper

Required

Required Reading for Week 3

The Dangers of Replication and a Solution
Gray, Helland, O'Neil, Shasha · ACM SIGMOD 1996, pp. 173–182

Why eager and lazy replication scale badly. Read sections 1–3.

Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases
Verbitski et al. · ACM SIGMOD 2017, pp. 1041–1052

"The log is the database." Read sections 1–3.

In Search of an Understandable Consensus Algorithm (Raft)
Ongaro, Ousterhout · USENIX ATC 2014, pp. 305–319

Safe leader election for failover. Read sections 1–2 and 5.1–5.3.

Bring a half-page note per paper: the problem, the key idea, and its link to this week.

Optional

Going Further

For students who want depth, or who are choosing a topic for the course project.

A Case for Redundant Arrays of Inexpensive Disks (RAID)

Patterson, Gibson, Katz · ACM SIGMOD 1988, pp. 109–116

Redundancy at the disk level: what it protects against, and what it does not.

Don’t Be Lazy, Be Consistent: Postgres-R

Kemme, Alonso · VLDB 2000, pp. 134–143

Eager replication made practical inside PostgreSQL using group communication.

Concurrency Control and Recovery in Database Systems

Bernstein, Hadzilacos, Goodman · Addison-Wesley 1987

Chapters 6–8: the theory of logging, recovery and replicated data.

In practice

RAID protects against a failed disk, but not against a DROP TABLE: the mistake is written to every disk at once. Only backups and the archived log can undo it.

Required

Update-Anywhere Replication Does Not Scale

Gray, J., Helland, P., O'Neil, P., & Shasha, D. (1996). The dangers of replication and a solution. In Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data (SIGMOD '96) (pp. 173–182). ACM. https://doi.org/10.1145/233269.233330

THE PROBLEM

Replicating data at many nodes looks like an easy way to gain availability and speed. Gray and colleagues ask what happens to waits, deadlocks and reconciliations when the number of nodes and the traffic grow.

KEY IDEAS

01 When: eager replication updates all copies in one transaction; lazy replication does it after commit

02 Who: group ownership lets any copy be updated; master ownership allows updates only at the owner

03 A simple analytic model counts waits, deadlocks and reconciliations as the number of nodes grows

Key term: Reconciliation — Repairing conflicting updates already committed at different copies, done by a program or a person

Required

Ten Times More Nodes, a Thousand Times More Deadlocks

WHAT TO TAKE AWAY

1. Eager replication: deadlocks grow with the cube of the nodes and the fifth power of transaction size
2. Lazy group replication turns waits into reconciliations, which grow just as fast
3. Proposed fix, two-tier replication: mobile nodes run tentative transactions, re-run on base nodes

THIS WEEK

Lesson 2-2's 'Synchronous vs Asynchronous Replication' slide is eager versus lazy. PostgreSQL's single primary is master ownership; you see it in Practice 2's optional replication demo.

LIMITATION

The model is simple: uniform access, no hotspots, message delays ignored. It predicts trends, not real rates, and its mobile setting is from 1996.

For your half-page note: PostgreSQL streaming replication has one primary. Which of Gray's four schemes is it, and why does it never need reconciliation?

Required

The Log Is the Database

Verbitski, A., Gupta, A., Saha, D., Brahmadesam, M., et al. (2017). Amazon Aurora: Design considerations for high throughput cloud-native relational databases. In Proceedings of SIGMOD '17 (pp. 1041–1052). ACM. https://doi.org/10.1145/3035918.3056101

THE PROBLEM

In the cloud, storage is spread over many nodes, so the network becomes the bottleneck. A mirrored MySQL writes each change many times: redo log, pages, double-write buffer, binlog. Amazon redesigns the engine around this.

KEY IDEAS

01 Six copies across three availability zones; a write needs 4 of 6 votes, a read needs 3 of 6

02 The volume is split into 10 GB segments, so a failed copy can be rebuilt in about 10 seconds

03 The engine sends only redo log records; storage nodes build the pages from them in the background

Key term: Quorum — The number of copies that must confirm a read or a write; read and write quorums must overlap

Required

Recovery and Backup Become Background Work

WHAT TO TAKE AWAY

1. In a 30-minute SysBench test, Aurora ran 35 times more transactions than mirrored MySQL
2. The database node made 7.7 times fewer I/Os per transaction, despite writing six copies
3. No replay at start-up: storage applies redo continuously and stages log and pages to Amazon S3

THIS WEEK

Connects Week 1's WAL to Lesson 2-3's 'Crash Recovery and Media Recovery' and RPO and RTO slides. In Practice 2 Part B you replay archived WAL yourself during point-in-time recovery.

LIMITATION

An industry paper on a closed service, with results from Amazon's own tests. Its six copies follow one log, so they also copy a mistaken DROP TABLE.

For your half-page note: Which Week 1 idea lets Aurora send only log records instead of pages? How does this change its RPO and RTO?

Required

Consensus Designed to Be Understood

Ongaro, D., & Ousterhout, J. (2014). In search of an understandable consensus algorithm. In Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14) (pp. 305–319). USENIX Association.

THE PROBLEM

Paxos dominated consensus for a decade, but students and system builders found it very hard to understand and to turn into working systems. Ongaro and Ousterhout design a consensus algorithm whose first goal is understandability.

KEY IDEAS

01 A replicated state machine: every server applies the same log of commands in the same order

02 Each server is a leader, a follower or a candidate; time is divided into numbered terms

03 Raft splits consensus into leader election, log replication and safety, each explained alone

Key term: Term — A numbered period of time that begins with an election and has at most one leader

Required

Majority Votes Give At Most One Leader per Term

WHAT TO TAKE AWAY

1. Each server votes once per term and a candidate needs a majority, so no term can have two winners
2. Random election timeouts, for example 150–300 ms, make split votes rare and end them quickly
3. An entry is committed once a majority stores it; a leader that sees a higher term steps down

THIS WEEK

Lesson 2-2's 'Failover: Promoting the Standby' and 'Split Brain' slides rely on it. In Practice 2's optional replication demo you promote the replica by hand; Patroni would decide via etcd.

LIMITATION

Raft assumes servers fail by stopping, not by lying. Without a majority the cluster accepts no writes, so two servers cannot survive one failure.

For your half-page note: Why must any two majorities of the same cluster overlap, and how does this prevent split brain during failover?

Optional

Many Cheap Disks Need Redundancy

Patterson, D. A., Gibson, G., & Katz, R. H. (1988). A case for redundant arrays of inexpensive disks (RAID). In Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data (SIGMOD '88) (pp. 109–116). https://doi.org/10.1145/50202.50214

THE PROBLEM

In the 1980s, processors and memory grew fast, but large expensive disks improved slowly. Arrays of small PC disks promise more I/O, yet with 100 disks one fails about every two weeks. The authors look for a design.

KEY IDEAS

01 Arrays of inexpensive disks can beat one large disk on I/O rate, cost per megabyte and power

02 Group the disks with extra check disks, so the data of one failed disk can be rebuilt

03 Five RAID levels, from mirrored disks (level 1) to parity spread across all disks (level 5)

Key term: Mean time to failure — Average operating time before a device fails; an array's MTTF falls as more disks are added

Optional

Redundancy Protects Against a Disk, Not a Mistake

WHAT TO TAKE AWAY

1. Without redundancy, 100 small disks give an array MTTF of 300 hours, under two weeks
2. Level 5 spreads data and parity over all disks, so several small writes can run in parallel
3. The reliability figures assume independent failures, which an earthquake or power surge breaks

THIS WEEK

Supports Lesson 2-1's 'Common Causes of Data Loss' and Lesson 2-2's 'Replication is not a backup'. The DELETE in Practice 2 Part A would reach every disk of a RAID array at once.

LIMITATION

A 1988 paper about magnetic disks and their prices. It leaves out cabling, controllers and software, and says nothing about human error.

To think about: RAID 5 survives one failed disk. Which causes of data loss from Lesson 2-1 does it not cover, and what does cover them?

Optional

Eager Replication Can Work on a Cluster

Kemme, B., & Alonso, G. (2000). Don't be lazy, be consistent: Postgres-R, a new way to implement database replication. In Proceedings of the 26th International Conference on Very Large Data Bases (VLDB 2000) (pp. 134–143). Morgan Kaufmann.

THE PROBLEM

After Gray et al., most designers believed that eager, update-everywhere replication was impractical. Lazy schemes give up consistency or depend on a primary copy. Kemme and Alonso want eager replication that performs well.

KEY IDEAS

01 Run each transaction locally on shadow copies, then send all its writes in one write-set message

02 Group communication delivers the write sets to all sites in the same total order

03 Each site takes all write locks of a write set in one atomic step, so these transactions never deadlock

Key term: Total order multicast — A group communication service that delivers every message to all sites in the same order

Optional

Built Into PostgreSQL and Tested on 15 Nodes

WHAT TO TAKE AWAY

1. From one to five servers, distributed locking slowed sharply while Postgres-R stayed stable
2. Response times fell as nodes were added, and abort rates never exceeded 5% in the tests
3. With mostly read-only queries, 15 servers handled up to 225 transactions per second

THIS WEEK

A reply to Gray et al. for Lesson 2-2's 'Synchronous vs Asynchronous Replication'. The replication demo of Practice 2 shows today's PostgreSQL choice instead: one primary, streaming WAL.

LIMITATION

The tests used PostgreSQL's no-flush option, which forces nothing to disk and breaks durability. Only 15 nodes and small tables were tested.

To think about: Postgres-R lets every site accept updates. Why do you think PostgreSQL today still replicates from a single primary?

Optional

The Formal Theory of Recovery and Replication

Bernstein, P. A., Hadzilacos, V., & Goodman, N. (1987). Concurrency control and recovery in database systems (Chs. 6–8). Addison-Wesley.

THE PROBLEM

Recovery and replication algorithms were often explained one system at a time. Bernstein, Hadzilacos and Goodman define failures, logs and correctness precisely, so that any such algorithm can be checked.

KEY IDEAS

01 Chapter 6: the Undo and Redo Rules and four algorithm families, from undo/redo to no-undo/no-redo

02 Chapter 7: atomic commitment, two-phase commit, its blocking problem, and three-phase commit

03 Chapter 8: one-copy serializability, available copies and quorum consensus for replicated data

Key term: One-copy serializability — A replicated execution that is equivalent to a serial execution on a database with one copy of each item

Optional

Undo, Redo and Commit Rules Behind Every Log

WHAT TO TAKE AWAY

1. The Undo Rule is write-ahead logging; the Redo Rule puts committed values in stable storage
2. In two-phase commit, a participant that voted Yes is blocked if the coordinator fails
3. Write-all replication stops when a site is down; quorums of votes let the system keep working

THIS WEEK

The theory behind Lesson 2-3's 'Crash Recovery and Media Recovery' and 'Standard Recovery: Restore, Then Roll Forward'. Practice 2 Part B's PITR is media recovery from an archive.

LIMITATION

A 1987 textbook: formal and dense, with proofs and exercises. Its examples predate group communication, Raft-style consensus and cloud storage.

To think about: Why can two-phase commit block, and how does a majority-based protocol such as Raft avoid the same problem?

Reading Questions

Try to answer each question in two or three sentences before the next class.

Q1	Why do waits and deadlocks grow so quickly with eager replication, and what does lazy replication give up?	Gray et al.
Q2	Why does Aurora send only redo log records over the network, and what does this change for backup and recovery?	Verbitski et al.
Q3	How does Raft guarantee that a cluster never has two leaders at once?	Ongaro, Ousterhout
Q4	A bank needs RPO = 0 and RTO under one minute. Which ideas from this week's papers and lessons would you combine?	All papers

WEEK 2 · WRAP-UP

This Week’s Work

Practice 2

Lab report

Backup, restore and (bonus) point-in-time recovery. One to two pages, 10 points.

Due: before Week 3

Quiz 2

Backup and recovery

Lessons 2-1, 2-2 and 2-3: multiple choice plus one short design question.

Due: before Week 3

Reading

Required papers

Gray et al., Aurora and Raft. A half-page note for each paper.

Bring to Week 3

Next week: many users at the same time. How does the DBMS stop their transactions from damaging each other?